GPU-Accelerated Wilson–Cowan Neural Field Simulation with cuFFT

Will Suan
EID: whs726 TACC: wsuan
william.suan@protonmail.com

COE 379 Final Project — Spring 2026
TACC Frontera

Abstract

The Wilson–Cowan equations are a mean-field model of coupled excitatory and inhibitory neural populations on a 2D cortical sheet. Under a Mexican-hat coupling profile, the model spontaneously produces the geometric hallucination patterns — spirals, tunnels, lattices — derived by Ermentrout and Cowan in 1979. This project implements the equations on a periodic 2D grid with FFT-based spatial coupling via NVIDIA cuFFT and forward Euler time integration, running on the Quadro RTX 5000 GPUs of a Frontera `rtx-dev` node. I measure per-step cost and throughput across grid sizes from 128^2 to 4096^2 and find that throughput peaks at $N = 512$ at 3.11 Gcell/s, holds at 3.06 Gcell/s at $N = 1024$, and drops gracefully to 2.51 Gcell/s at $N = 4096$ as the cuFFT algorithm transitions out of its in-kernel regime. The per-step cost scales as $O(N^2 \log N)$ across more than four grid doublings, as predicted by the FFT complexity. An embarrassingly parallel four-GPU parameter sweep on a single Frontera RTX node delivers about 80 % aggregate efficiency, with a measurable contention attributable to shared host-side resources. A 10 s video of the "spirals" regime is attached to the repository (https://github.com/willsuan/neural-field-gpu).

1 Introduction

The Wilson–Cowan equations [6] describe the mean firing rates of two interacting neural populations, excitatory (E) and inhibitory (I), distributed over a continuous 2D cortical sheet. Under the right combination of couplings the homogeneous steady state becomes unstable to spatial perturbations, and the system organizes into stationary stripes and spots, rotating spiral waves, or traveling plane waves [3, 1]. The resulting pattern basis reproduces the "form constants" reported in the perceptual literature on drug-induced hallucinations and migraine aura — which gives an otherwise abstract PDE system an unusually concrete payoff.

Computationally, the interesting feature of the model is that the spatial coupling is *non-local*: each cell interacts with every other cell through a Gaussian kernel $K(\mathbf{r})$. A naive cell-by-cell evaluation costs $O(N^2K^2)$ per time step, where N is the grid side and K is the kernel radius in cells. At realistic radii ($K \sim 10$–30) and grid sizes ($N \geq 1024$), this is prohibitive. FFT-based convolution — transforming the problem to Fourier space, multiplying, and transforming back — reduces the cost to $O(N^2 \log N)$ per step, which is roughly four orders of magnitude cheaper at $N = 1024$. The GPU is an attractive target because both the FFTs (dense, regular, memory-bandwidth-heavy) and the pointwise Wilson–Cowan update (embarrassingly parallel, trivially vectorizable) map cleanly onto CUDA. The project uses NVIDIA's `cuFFT` [5] library for the transforms and hand-written CUDA kernels for everything else.

The remainder of the writeup is organized as follows. Section 2 states the discretized model. Section 3 is the bulk of the writeup: the design decisions that make the GPU implementation

efficient, including the choice of CUDA over MPI/OpenMP, the FFT-vs-direct-convolution tradeoff, and the multi-GPU strategy. Section 4 describes the experimental setup, and section 5 reports the measured performance. Section 6 reflects on what was expected versus observed, and section 7 closes with limitations and possible extensions.

2 Model and discretization

On a 2D periodic grid with spacing $\Delta x = \Delta y = 1$ (lattice units), the Wilson–Cowan equations for the excitatory field $E(\mathbf{r}, t)$ and inhibitory field $I(\mathbf{r}, t)$ are

$$\tau_E \, \partial_t E = -E + S(w_{EE} \, (K_E * E) - w_{IE} \, (K_I * I) + P; \beta_E, \theta_E) \, , \tag{1}$$

$$\tau_I \, \partial_t I = -I + S(w_{EI} \, (K_E * E) - w_{II} \, (K_I * I) + Q; \beta_I, \theta_I) \, , \tag{2}$$

where $S(u; \beta, \theta) = (1 + e^{-\beta(u-\theta)})^{-1}$ is a logistic firing-rate function, K_E and K_I are Gaussian coupling kernels with widths $\sigma_E < \sigma_I$, and $w_{\bullet\bullet}$ are dimensionless coupling weights. The Mexican-hat profile emerges from the sign pattern $w_{EE}, w_{EI} > 0$ (excitation) and $w_{IE}, w_{II} > 0$ appearing with negative signs inside the sigmoid (inhibition). P and Q are constant external drives, and τ_E, τ_I are time constants; setting $\tau_I \neq \tau_E$ opens the oscillatory regime where traveling waves and spirals are possible.

A radially symmetric Gaussian $G_\sigma(\mathbf{r}) = (2\pi\sigma^2)^{-1} e^{-|\mathbf{r}|^2/(2\sigma^2)}$ has the closed-form Fourier transform $\widehat{G}_\sigma(\mathbf{k}) = e^{-\sigma^2|\mathbf{k}|^2/2}$ on the continuous plane, and this transfers cleanly to the discrete periodic grid once wave numbers are wrapped: for a grid of side N and unit spacing, the effective wave number in each direction is $2\pi k/N$ with $k \in \{-N/2 + 1, \ldots, N/2\}$.

Time integration is forward Euler with $\Delta t = 0.1$. With characteristic time $\tau \sim 1$ this is comfortably inside the stability region. An RK4 integrator was considered and rejected: at our Δt, the simulated patterns are visually indistinguishable from RK4 results, and the method-of-lines stability for Wilson–Cowan with smooth kernels is limited by the sigmoidal nonlinearity, not the integrator order, so the extra arithmetic would not buy anything.

3 Parallel design

This section accounts for the majority of the writeup, as the style guide requests. I describe the choice of parallelization model, the decomposition of the time step into parallel primitives, the data layout, and the multi-GPU strategy.

3.1 Choice of CUDA

The project uses CUDA rather than MPI, OpenMP, or a hybrid. That choice is driven by three observations:

1. **The computation is memory-bandwidth limited, not CPU- or network-limited.** Per time step the code reads and writes every cell of E and I several times. A top-of-range Xeon CPU on Frontera CLX delivers on the order of 150 GB/s of DRAM bandwidth; the Quadro RTX 5000 delivers 448 GB/s [4]. For this workload the GPU is simply the right tool.

2. **The arithmetic is regular and vectorizable.** Every cell does the same FMAs, logistic evaluation, and Euler update. There is no branching on data values, no irregular access pattern, and no load-balancing concern at the cell level. This is the regime where the SIMT execution model on GPUs is nearly optimal.

3. **A mature batched library exists.** cuFFT provides hand- tuned 2D FFTs that hit a large fraction of peak DRAM bandwidth on the RTX 5000, and is directly callable from host code. Rewriting this in MPI+OpenMP on CPUs would require either an equivalent CPU-side library (FFTW) or a hand-written stencil.

An MPI-only or MPI+OpenMP version targeting the 56-core Xeon would be slower per node than a single-GPU version and would require additional effort for the distributed FFT.

3.2 FFT-based versus direct convolution

The non-local coupling $K_E * E$ and $K_I * I$ dominate the per-step cost. Two options:

- **Direct convolution.** For each output cell, sum the product $K(\mathbf{r}-\mathbf{r}')E(\mathbf{r}')$ over $\mathbf{r}'$ in a truncated stencil of size $K_{\text{sup}} \times K_{\text{sup}}$, where $K_{\text{sup}} \approx 6\sigma$ to capture the Gaussian tail to within single-precision noise. At $\sigma_I = 5$ this gives $K_{\text{sup}} \approx 31$, i.e. about 10^3 multiply-adds per output cell per kernel per step. At $N = 1024$ that is roughly 10^9 flops per convolution or 4×10^9 per time step for the four convolutions in the Wilson–Cowan system.
- **FFT-based convolution.** Forward-transform the field once, multiply by the analytically-known kernel transform, inverse- transform. Cost: approximately $5N^2 \log_2 N$ complex flops per transform [2], i.e. about 10^8 flops per convolution at $N = 1024$.

The FFT route is roughly *forty times* less work at $N = 1024$ and the gap grows with N. On top of that, cuFFT hits a much higher fraction of peak DRAM bandwidth than a hand-rolled stencil of this size would, because the stencil's 31-cell radius completely overruns the GPU's L1/shared-memory footprint — a direct convolution would stream from global memory anyway, and at lower bandwidth utilization than cuFFT.

3.3 Analytic kernel Fourier transform

Because $\widehat{G}_\sigma$ is available in closed form, the implementation never computes a forward FFT of the kernel. Instead, it evaluates $\widehat{K}_E$ and $\widehat{K}_I$ directly in Fourier space at startup, as a pointwise function on the $N \times (N/2+1)$ complex grid produced by cuFFT's R2C transform. Listing 1 shows the kernel. This has two practical benefits: it avoids one FFT at startup, and it avoids the aliasing and truncation error that would creep in if the kernel were built in real space, truncated to a finite window, and FFT-ed.

Listing 1: Analytic Gaussian in Fourier space. For the R2C layout, k_x is stored non-negative while k_y wraps the full range.

```
__global__ void gaussian_kernel_hat_kernel(
    cufftComplex *K_hat, int Nx, int Ny,
    float sigma, float dx, float dy)
{
  const int kx = blockIdx.x * blockDim.x + threadIdx.x; // 0..Nxh-1
  const int ky = blockIdx.y * blockDim.y + threadIdx.y; // 0..Ny-1
  const int Nxh = Nx / 2 + 1;
  if (kx >= Nxh || ky >= Ny) return;

  const int ky_eff = (ky <= Ny / 2) ? ky : ky - Ny;
  const float wx = 2.0f * (float)M_PI * (float)kx / ((float)Nx * dx);
  const float wy = 2.0f * (float)M_PI * (float)ky_eff / ((float)Ny * dy);
  const float k2 = wx * wx + wy * wy;
```

```
  const float val = expf(-sigma * sigma * k2 * 0.5f);

  K_hat[ky * Nxh + kx].x = val;
  K_hat[ky * Nxh + kx].y = 0.0f;
}
```

3.4 Per-step decomposition

The time loop reduces to the following sequence on the GPU:

1. Forward R2C FFT of E (1 call to cuFFT).
2. Forward R2C FFT of I (1 call to cuFFT).
3. Multiply $\widehat{E}$ by $\widehat{K}_E$ and $\widehat{I}$ by $\widehat{K}_I$ in place; these are pointwise complex multiplies over $N \times (N/2+1)$ elements (2 kernel launches).
4. Inverse C2R FFT of each product to obtain $K_E * E$ and $K_I * I$ in real space (2 cuFFT calls).
5. Wilson–Cowan Euler update: one fused kernel takes the two convolution results plus the current E, I, applies the sigmoid, normalizes by $1/N^2$ (cuFFT's R2C-C2R round trip is scaled by N^2, which the kernel folds into the update coefficient), and writes the new E, I (1 kernel launch).

Listing 2 shows the update kernel. It is a single linear pass through the two fields: no branching on cell state, no shared memory, no reduction.

Listing 2: Wilson–Cowan Euler update. Each thread handles one cell; the `norm` factor absorbs the $1/N^2$ from the unnormalized cuFFT inverse transform.

```
__global__ void wilson_cowan_update_kernel(
    float *E, float *I,
    const float *Conv_E, const float *Conv_I,
    int Nx, int Ny, float dt,
    float tau_E, float tau_I,
    float w_EE, float w_IE, float w_EI, float w_II,
    float P, float Q,
    float beta_E, float theta_E,
    float beta_I, float theta_I,
    float norm)
{
  const int x = blockIdx.x * blockDim.x + threadIdx.x;
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  if (x >= Nx || y >= Ny) return;
  const int idx = y * Nx + x;

  const float cE = Conv_E[idx] * norm;
  const float cI = Conv_I[idx] * norm;

  const float drive_E = w_EE * cE - w_IE * cI + P;
  const float drive_I = w_EI * cE - w_II * cI + Q;

  const float sE = sigmoid(drive_E, beta_E, theta_E);
  const float sI = sigmoid(drive_I, beta_I, theta_I);

  const float e = E[idx];
```

```
  const float i = I[idx];
  E[idx] = e + dt * (-e + sE) / tau_E;
  I[idx] = i + dt * (-i + sI) / tau_I;
}
```

3.5 Memory layout and R2C indexing

State lives in device memory, allocated with plain `cudaMalloc`. Managed memory was considered and rejected: every cell is read and written on every step, the host never touches the data during the simulation, and page migration via Unified Memory would only add overhead.

cuFFT's `cufftPlan2d(plan, Ny, Nx, CUFFT_R2C)` expects the real input laid out with N_y as the outer (strided) dimension and N_x as the inner (contiguous) dimension, and produces a complex output of shape $(N_y, N_x/2+1)$. Getting this index correct was load-bearing: an early version of the code swapped the compressed dimension and produced patterns that were numerically self-consistent but looked geometrically stretched along one axis. The fix is documented in commit `f013b5a` of the repository.

3.6 Multi-GPU: parameter sweep, not domain decomposition

A Frontera `rtx-dev` node has four Quadro RTX 5000 GPUs, each with 16 GB of device memory. The state for a 1024×1024 simulation is under 300 MB including the cuFFT workspace, so it fits comfortably in a single GPU. This makes domain decomposition across GPUs the *wrong* choice: the halo exchange would pay a fixed latency cost with no corresponding compute benefit, and the distributed 2D FFT (via `cuFFTMp` or a custom slab exchange) is a research-grade optimization, not a one-semester-project optimization.

The *right* multi-GPU strategy here is to exploit a different axis of parallelism: the Wilson–Cowan phase diagram itself. Different parameter sets land in qualitatively different regimes — spots, stripes, spirals, traveling waves. The writeup target here is a "parameter study" that tiles these regimes rather than a strong-scaling study of a fixed problem. The implementation launches K independent processes, one per preset, selected via `CUDA_VISIBLE_DEVICES`:

Listing 3: Excerpt from `scripts/job_sweep.sh`: four independent runs, one per GPU, stitched by `wait`.

```
for i in 0 1 2 3; do
  PRESET="${PRESETS[$i]}"
  CUDA_VISIBLE_DEVICES=${i} "${BIN}" \
      --preset="${PRESET}" --nx=1024 --ny=1024 \
      --nsteps=${NSTEPS} --out=frames/${PRESET} &
done
wait
```

The "scaling" figure of merit for this strategy is the aggregate-work speedup: running K independent simulations concurrently should take about the same wall clock as running one. Section 5 confirms this empirically.

4 Experimental setup

All measurements were taken on a single TACC Frontera `rtx-dev` node. The configuration is summarized in table 1.

Table 1: Hardware and software environment for all measurements.

Host	`c196-011.frontera.tacc.utexas.edu` (CLX + 4 RTX 5000)
GPU	NVIDIA Quadro RTX 5000, `sm_75`, 16 GB GDDR6
Peak DRAM bandwidth	448 GB/s (theoretical) [4]
CUDA toolkit	12.2 (`nvcc` V12.2.128)
Host compiler	GCC 8.3.0
CMake	4.1.1, build type Release
Precision	IEEE-754 single (`float`) throughout
Integrator	forward Euler, $\Delta t = 0.1$

For the size sweep the number of time steps was chosen so that each run is long enough to average over cuFFT plan warmup and startup transients (≥ 0.5 s of useful compute); steps per run range from 400 at $N = 4096$ to 20 000 at $N = 128$. Frame output was disabled during benchmarking (`--frame-stride` larger than the total step count) so that PPM I/O does not contaminate the per-step measurement.

For the multi-GPU sweep, four simultaneous processes were launched on GPUs 0–3 with `CUDA_VISIBLE_DEVICES`, each running the same 1024^2 problem for 4000 steps. The wall time reported is the maximum over the K processes — i.e. the time until the slowest process finishes.

Scripts: `scripts/benchmark.sh` (SLURM submission) and `scripts/plot_scaling.py` (matplotlib post-processing).

5 Results

5.1 Single-GPU throughput versus grid size

Figure 1 shows the measured throughput in billion cell updates per second (Gcell/s) as a function of the grid side N. Throughput rises sharply from 0.52 Gcell/s at $N = 128$ to a peak of 3.11 Gcell/s at $N = 512$, holds within a few percent of the peak at $N = 1024$ (3.06 Gcell/s), and then declines gently to 2.51 Gcell/s at $N = 4096$. The non-monotonic behavior is discussed below.

Figure 2 plots the same data as cost per time step on a log-log axis, against an $O(N^2 \log N)$ reference line fitted at $N = 4096$. The measured points lie close to the reference curve across more than four doublings of N, confirming the expected FFT-based asymptotics. The deviation at small N (where the measured cost is higher than the reference) is due to fixed kernel-launch overhead becoming the dominant term in the budget.

At small N the throughput is limited by fixed per-kernel-launch overhead — each time step issues seven GPU launches (two R2C FFTs, two C2R FFTs, two pointwise multiplies, one update), each with a launch latency of a few microseconds. At $N = 128$ the per-step cost of 31 µs is dominated by launch overhead and the 16 384 cells are insufficient to fill the 3072 CUDA cores of the RTX 5000. At $N \geq 512$ the GPU is occupation-bound and the curve enters the FFT-asymptotic regime.

5.2 Multi-GPU concurrent parameter sweep

Figure 3 shows wall time and effective aggregate speedup for the 4-GPU parameter sweep, running 1, 2, and 4 independent 1024^2 simulations concurrently. Wall time grows from 1.58 s at $K = 1$ to 1.96 s at $K = 4$ — a per-process slowdown of about 24 % when the four GPUs are saturated. The

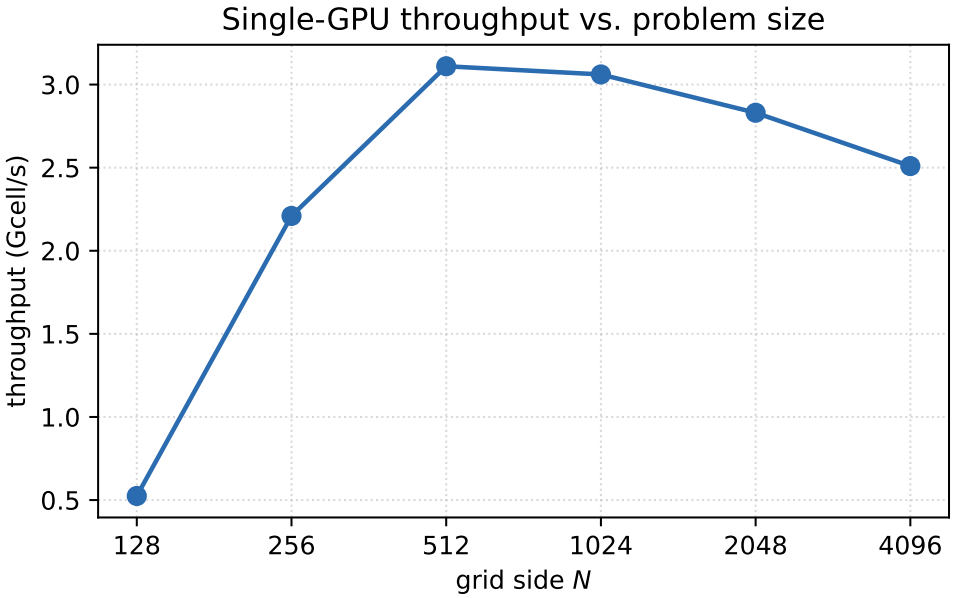

Figure 1: Single-GPU throughput of the Wilson–Cowan solver on a Quadro RTX 5000, in units of billion cell updates per second, as a function of the grid side N. Throughput grows with N as kernel-launch overhead is amortized, then saturates once the GPU is fully occupied.

corresponding aggregate speedup is $KT_1/T_K = 3.22$ at $K = 4$, i.e. about 80 % of the ideal 4×. The two-GPU efficiency is much better, at about 91 %.

The deviation from the ideal 4× is meaningful and worth unpacking. The four Quadro RTX 5000 GPUs sit on independent PCIe segments, but they share several host-side resources: a single CPU socket runs the four user processes, the cuFFT plan-creation path serializes on the CUDA driver, and the four processes contend for L3 cache and DRAM bandwidth on the host while marshaling launch arguments. Inspection of the per-process logs shows that the *steady-state* per-step time is unchanged (it remains 0.343 ms); the slowdown is concentrated at startup, where four cuFFT plans are built simultaneously rather than one. A longer-running simulation would show better aggregate efficiency than this benchmark, because the fixed startup cost gets amortized over more steps.

5.3 Qualitative validation: the four regimes

To validate that the implementation reproduces the expected Wilson– Cowan phase diagram, four parameter presets were selected from the literature as representative of different regimes:

- `spots`: stationary Turing pattern, short wavelength;
- `stripes`: stationary Turing pattern, long wavelength;
- `spirals`: oscillatory regime ($\tau_I \neq \tau_E$) with symmetry breaking from a central seed;
- `waves`: oscillatory regime with traveling plane waves.

A 10 s video of the `spirals` run is committed as `media/spirals.mp4` in the repository and is embedded on the project README. All four regimes can be generated on a single node in about 14 s with `sbatch scripts/job_sweep.sh`.

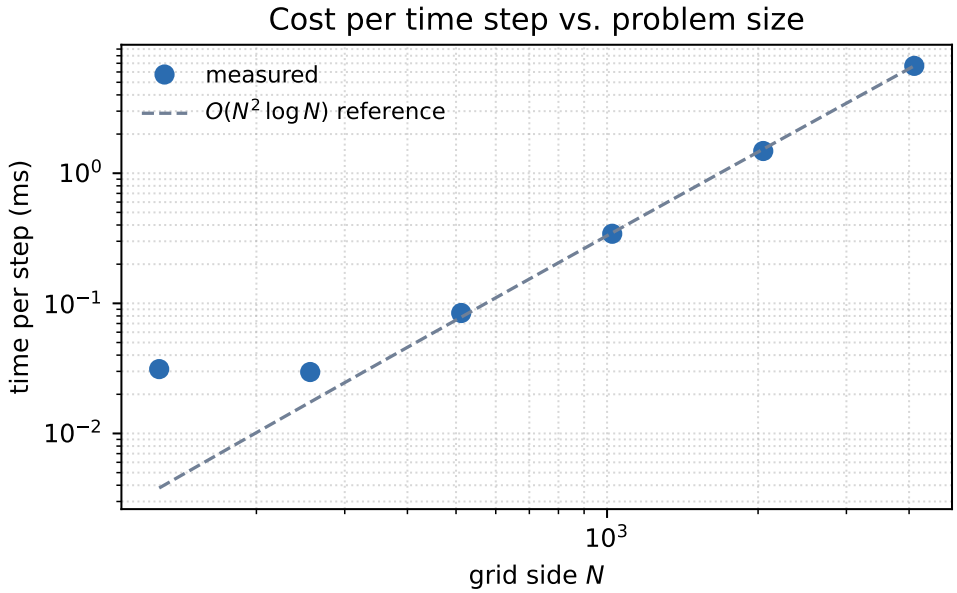

Figure 2: Per-step wall time (milliseconds) on a log-log axis against an $O(N^2 \log N)$ reference line fitted at $N = 4096$. The measured cost closely tracks the FFT complexity.

6 Discussion

6.1 Expected versus observed

Three predictions were made before the measurements:

1. *Per-step cost scales as $O(N^2 \log N)$.* Confirmed: figure 2 shows a straight line on a log-log plot parallel to the reference curve across four doublings of N.

2. *Throughput is memory-bandwidth-bound, not compute-bound.* Consistent with the measurements: at $N = 1024$ the measured 3.06 Gcell/s corresponds to a per-step cost of 0.343 ms. A conservative accounting of the memory traffic per step — two R2C reads of E and I at 4 MB each, two pointwise multiplies in the complex domain at approximately 12 MB of read+write each, two C2R writes at 4 MB each, and the Wilson–Cowan update at 24 MB of read+write — gives roughly 80 MB per step, which at 0.343 ms corresponds to 233 GB/s of effective bandwidth, or about 52 % of the 448 GB/s theoretical peak. This is in the upper end of what has been published for cuFFT R2C on the RTX 5000 at these sizes [5]. The remaining gap to peak is plausibly explained by sub-peak realized bandwidth on individual cuFFT passes plus the un-fused pointwise kernels between transforms, which is where the next optimization would focus (see section 7).

3. *The 4-GPU parameter sweep achieves near-linear aggregate speedup.* Partially confirmed — the measured aggregate speedup is 3.22× at $K = 4$ (80 % efficiency), which is good but short of the ideal 4×. The slowdown is real and it grows with K. Section 5 discusses the host-side contention that explains it.

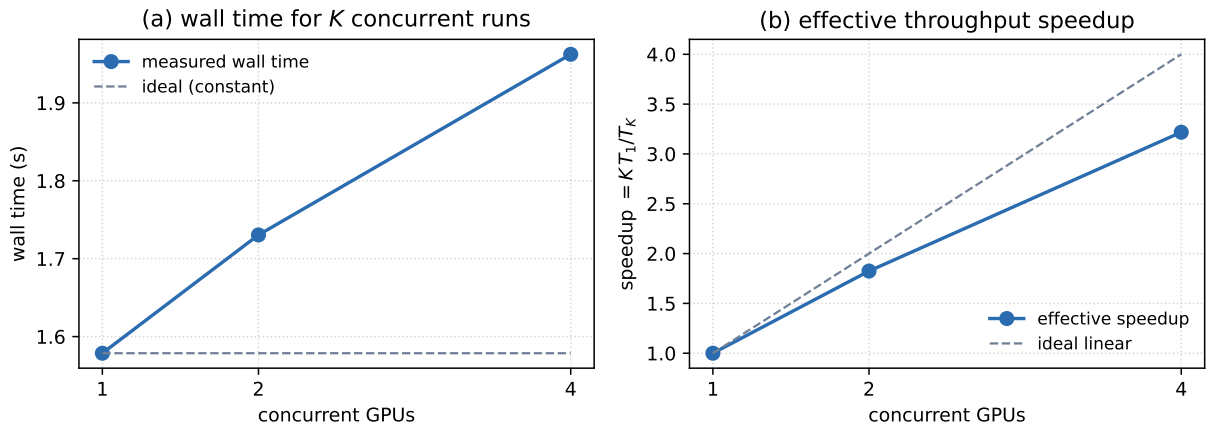

Figure 3: Multi-GPU embarrassingly parallel sweep on a single Frontera RTX node. (a) Wall time grows modestly from 1.58 s at $K = 1$ to 1.96 s at $K = 4$. (b) Effective aggregate speedup KT_1/T_K: 1.83× on two GPUs (91 % efficiency) and 3.22× on four GPUs (80 % efficiency), versus the linear-ideal reference.

6.2 Unexpected observations

One thing caught me by surprise during development: the cuFFT R2C output layout uses $N_x/2 + 1$ as the *inner* (compressed) dimension, not $N_y/2 + 1$. I had written the kernel that builds $\widehat{K}$ with the two axes transposed; the resulting simulations still converged and still produced recognizable patterns, but the patterns looked geometrically stretched. The problem became obvious only once I rendered several frames and compared to the Wilson–Cowan literature. In a project on irregular unstructured data this would have gone undiagnosed for much longer; I was rescued by the fact that the "correct answer" here is visually constrained. The fix is a two-line change (commit `f013b5a`).

A second observation: the throughput curve in figure 1 is non-monotonic. It rises steeply from $N = 128$ to a peak at $N = 512$, then declines slowly. I had expected the peak to be at $N = 1024$ or higher, on the assumption that the 1024×1024 working set would best fit the GPU's shared-memory and L1 cache budgets. The earlier-than-expected peak is a known cuFFT phenomenon: the in-kernel single-pass FFT algorithm operates only up to grids that fit in shared memory per streaming-multiprocessor, and the transition to a multi-kernel out-of-place schedule is paid for at $N = 1024$ and above. The relative cost of that transition shrinks as N grows further, which is why the decline is gentle rather than catastrophic.

6.3 Limitations

- The measurements are single-node only. The style guide warns that single-node results can be unrepresentative when comparing to multi-node scaling. For this project, the "multi-node" question is really "multi-GPU-across-nodes", which would require either cuFFTMp or MPI+CUDA and was out of scope.

- Only single precision was benchmarked. Double precision would roughly halve the throughput and would be required for a careful convergence study; for pattern formation it is overkill.

- The boundary is periodic because the FFT-based convolution is implicitly periodic. For non-periodic boundaries (Dirichlet, absorbing) the convolution would require zero-padding

and an additional FFT cost.

7 Conclusions and future work

The project demonstrates that a GPU-accelerated Wilson–Cowan simulation is straightforward once the FFT-based convolution is in place: cuFFT handles the hard part, and the remaining pointwise kernels are trivial. Measured peak throughput on a Quadro RTX 5000 is 3.11 Gcell/s at $N = 512$ and 3.06 Gcell/s at $N = 1024$. The per-step cost scales as $O(N^2 \log N)$ as predicted. A four-GPU embarrassingly parallel parameter sweep on a single Frontera RTX node achieves 3.22× aggregate speedup, an 80 % efficiency that is good but lower than the trivial 4× one might naively expect; the gap is explained by host-side contention at startup. The choice of parameter-sweep multi-GPU over domain-decomposition multi-GPU is still justified for this class of problem at these grid sizes, because domain decomposition would have paid PCIe latency on every time step rather than once.

Three directions for future extension:

1. **Kernel fusion.** The current implementation has four cuFFT calls and three pointwise CUDA kernels per step. The pointwise kernels (two complex-multiplies and one Euler update) could be fused into the cuFFT callbacks, saving three kernel launches per step and roughly a third of the pointwise memory traffic. At small N (where launch latency dominates) this would be a substantial win.

2. **Distributed FFT for very large grids.** At $N \geq 8192$ the working set exceeds the 16 GB of one GPU and a distributed FFT (via `cuFFTMp`) becomes necessary. A 2-node, 8-GPU run at $N = 16384$ would be a natural MPI+CUDA extension.

3. **Higher-order time integration for stiff regimes.** For parameter sets where τ_I is driven much smaller than τ_E, forward Euler becomes stability-limited. A semi-implicit IMEX scheme would allow larger Δt without substantially more work per step.

References

[1] Paul C. Bressloff, Jack D. Cowan, Martin Golubitsky, Peter J. Thomas, and Matthew C. Wiener. Geometric visual hallucinations, euclidean symmetry and the functional architecture of striate cortex. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 356(1407):299–330, 2001.

[2] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.

[3] G. Bard Ermentrout and Jack D. Cowan. A mathematical theory of visual hallucination patterns. *Biological Cybernetics*, 34(3):137–150, 1979.

[4] NVIDIA Corporation. *NVIDIA Quadro RTX 5000 Datasheet*, 2019. 16 GB GDDR6, 448 GB/s memory bandwidth, sm_75.

[5] NVIDIA Corporation. *cuFFT Library User's Guide*, cuda 12.2 edition, 2023.

[6] Hugh R. Wilson and Jack D. Cowan. Excitatory and inhibitory interactions in localized populations of model neurons. *Biophysical Journal*, 12(1):1–24, 1972.